

ELEC50001 – Circuits and Systems

SAMPLE PAPER 2

SOLUTIONS

Answer ALL questions.

There are THREE questions on the paper.

Question ONE counts for 50% of the marks, other questions 25% each

Time allowed: 2 hours

1. *Figure 1.1* shows an inverting amplifier circuit constructed from an operational amplifier with a Gain-Bandwidth product of 1MHz that has a single power supply at 5V, two resistors R1 and R2, and a voltage source at 2.5V.

- (i) Write down an equation for the Gain of this amplifier. [2]
- (ii) Given that the amplifier has a gain of -14 and $R1 = 2.2k\Omega$, determine the value of R2. [2]
- (iii) A design is required to provide an overall gain of -200 for a signal with frequency component from 1kHz to 50kHz. Explain why the circuit in *Figure 1.1* is unable to achieve the specification. [2]
- (iv) Hence or otherwise, design an additional circuit using a second operational amplifier to meet the required specification. [4]

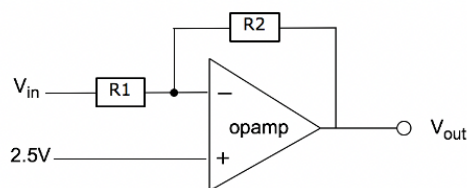
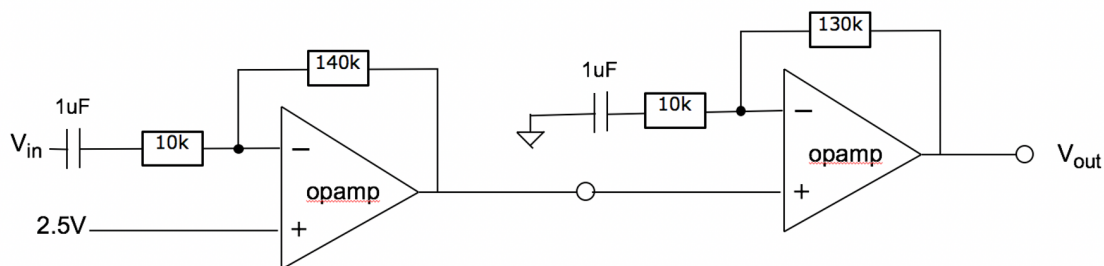


Figure 1.1

SOLUTION

- (i)
$$V_{out} = -\frac{R2}{R1}(V_{in} - 2.5) + 2.5$$
- (ii) For a gain of -14, $R2 = 14 \times R1 = 30.8k\Omega$
- (iii) The gain-bandwidth product is 1MHz. Therefore at 50kHz (max signal frequency), the maximum gain is only x20. Furthermore, the amplifier is DC coupled. It needs to add a DC blocking capacitor at the input.
- (iv) The 1kHz lower frequency requirement define the capacitor coupling we need. With circuits provided, the lower corner frequency is compatible with the 1kHz lower limit. See circuit below:



(b) Figure 1.2 depicts a circuit with four D-type flip-flops FF1-FF4 connected in series with an XOR gate.

(i) If FF1, FF2 and FF4 are initially reset, and FF3 is initially set, determine the output Q4:1 over 10 clock cycles. [2]

(ii) Implement this circuit in Verilog HDL with an interface defined in Figure 1.2. [5]

(iii) Given that $1 + x^3 + x^5$ is a primitive polynomial, design in the form of a schematic circuit diagram a linear-feedback shift register that produces a 5-bit pseudo-random binary number. [3]

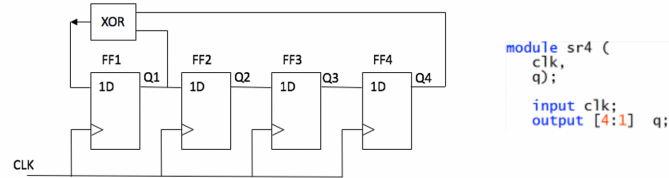


Figure 1.2

```
module sr4 (
    clk,
    q);
    input clk;
    output [4:1] q;
```

(b) This question tests students understanding of shift register and LFSR etc. It also tests the ability to use Verilog to specify a shift register circuit.

(i)

Q4:Q3:Q2:Q1	Q4^Q1
0100	0
1000	1
0001	1
0011	1
0111	1
1111	0
1110	1
1101	0
1010	1
1011	0
0110	0

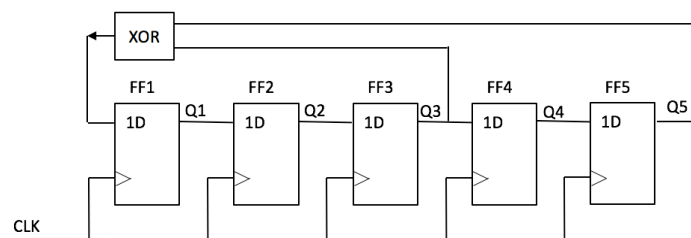
[2]

(ii)

```
module sr4 (
    clk,
    q);
    input clk;
    output [4:1] q;
    reg [4:1] sreg;
    initial sreg = 4'b0100;
    always @ (posedge clk)
    begin
        sreg[4:2] <= sreg[3:1];
        sreg[1] <= sreg[4] ^ sreg[1];
    end
    assign q = sreg;
endmodule
```

[5]

(iii)



[3]

- (c) Explain the principle of operation of a 12-bit pulse-width modulation (PWM) digital-to-analogue converter (DAC). Design in Verilog HDL a PWM DAC with an interface as defined in *Figure 1.3*.

[10]

```
module pwm_dac (clk, data_in, pwm_out);

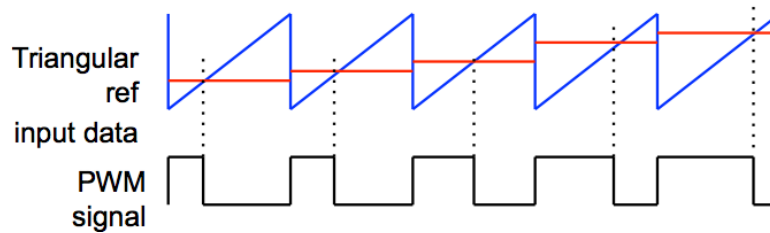
    input      clk;          // system clock
    input [11:0] data_in;    // input data for conversion
    output     pwm_out;      // PWM output

endmodule
```

Figure 1.3

- Q1 (c) Bookwork. This question tests student's understanding of PWM DAC which has been covered in the lectures.

Generate a triangular signal (in the form of a 12-bit counter) and compare the input value `data_in` to that of the counter value. Set `pwm_out` to be high if the counter value is lower than `data_in`, otherwise set it to low. The DAC output is the lowpass filtered version of the PWM signal.



```
module pwm_dac (clk, data_in, pwm_out);

    input      clk;          // system clock
    input [11:0] data_in;    // input data for conversion
    output     pwm_out;      // PWM output

    reg [11:0] count;        // internal 12-bit counter
    reg        pwm_out;

    initial count = 12'b0;

    always @ (posedge clk) begin
        count <= count + 1'b1;
        if (count > data_in)
            pwm_out <= 1'b0;
        else
            pwm_out <= 1'b1;
        end
    end

endmodule
```

(d) *Figure 1.4* shows a circuit with two D-flipflops FF1 and FF2 with setup and hold times of 3 ns and 2 ns respectively, and a clock-to-Q output delay of 2 ns. The clock signal CLK has a 1:1 mark-space ratio. The datapath is driven by logic_A, which has a propagation delay between 2 ns and 5 ns. The clock path is driven by logic_B, which has a propagation delay between 1 ns and 10 ns. Derive the maximum frequency of the signal CLK for reliable operation of this circuit.

[10]

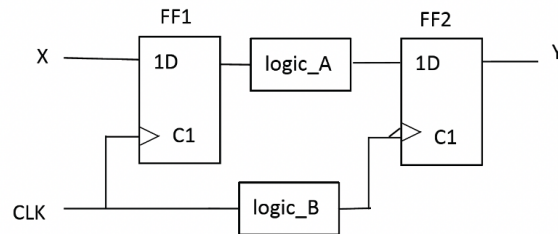
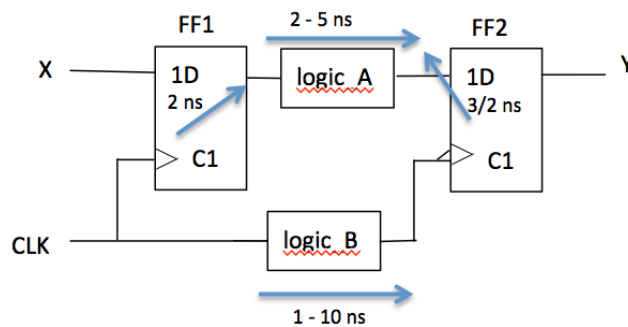


Figure 1.4

Q1 (d) This question tests student's ability to work out digital circuit timing constraints.



Setup time constraint:

$$tc-q(\max) + \text{logic_A}(\max) + t_{\text{setup}} < \frac{1}{2} T + \text{logic_B}(\min)$$

$$2 + 5 + 3 < \frac{1}{2} T + 1, \text{ therefore } T > 18\text{ns and } F_{\max}(\text{setup}) < 55.56\text{MHz}$$

Hold time constraint:

$$T + tc-q(\min) + \text{logic_A}(\min) > \frac{1}{2} T + \text{logic_B}(\max) + t_{\text{hold}}, \text{ therefore } T > 16\text{ns.}$$

Hold time is never violated.

[10]

- (e) Figure 1.5 shows a counter circuit where one of the flip-flops has an asynchronous reset input which forces Q2 to zero whenever M=1.

(i) For M=0, draw the state diagram for the circuit of Figure 1.5 assuming that all flip-flops are initially in the reset state. Draw a timing diagram showing the waveforms of the clock CLK, and the outputs Q0, Q1 and Q2 for one complete cycle of the counter.

[6]

(ii) Repeat (i) above, but for the case where M=1. By considering the output Q0, state the behaviour of this counter in relation to the control signal M.

[4]

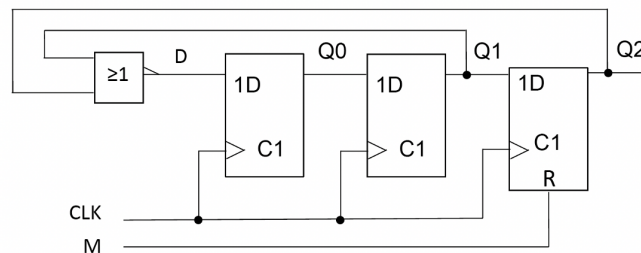


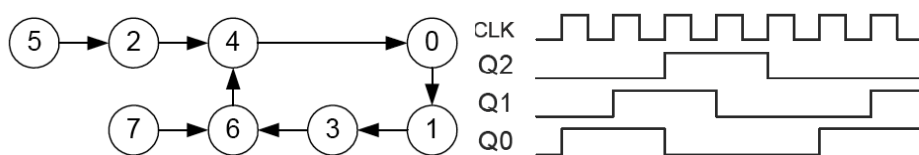
Figure 1.5

Q1 (e) This question tests student's ability to analyse a simple state machine.

(i) It is helpful to draw the transition table first:

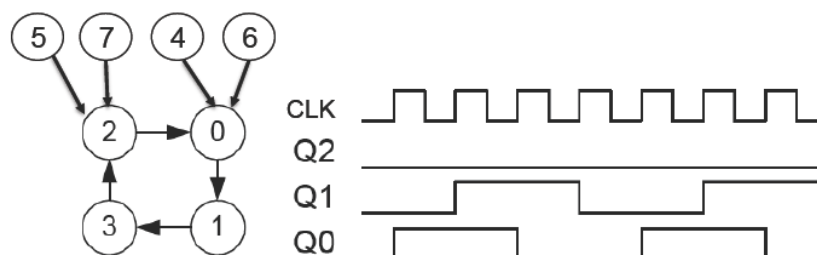
Q2	Q1	Q0		Next_Q2	Next_Q1	Next_Q0 (D)
0	0	0		0	0	1
0	0	1		0	1	1
0	1	0		1	0	0
0	1	1		1	1	0
1	0	0		0	0	0
1	0	1		0	1	0
1	1	0		1	0	0
1	1	1		1	1	0

Expect to show the illegal state too!



[6]

(ii)



If M = 0, CLK is divided by 5 otherwise CLK is divided by 4. Therefore it is a selectable divide by 4 or 5 counter.

[4]

2. *Figure 2.1* shows a finite state machine (FSM) with 1-bit input IN and 2-bit output OUT1:0, implemented using an 8 x 4-bit ROM and a 4-bit register. The input IN to the FSM drives the upper bit of the ROM address A2. The output of the ROM D3:0 is connected to the data input of the register. The lower 2 bits of the ROM data D1:0 provide the next state value, which are feedback to the lower 2 bits of the ROM address A1:0 through the state register S1:0. The upper 2 bits of the ROM data D3:2 provide the FSM output value. The reset input RST clears the register.

Figure 2.2 shows the state diagram of the FSM.

- (a) Determine the contents of the ROM in order to implement this FSM. [8]
- (b) A Logic Element (LE) in Intel's MAX10 FPGA device consists of a 4-input lookup table and a register. Design the SSM using LEs in the Cyclone II device using *one-hot state encoding*. Estimate how many LEs are required to implement the FSM. [10]
- (c) Write a synthesizable Verilog module to describe the FSM in (b). [7]

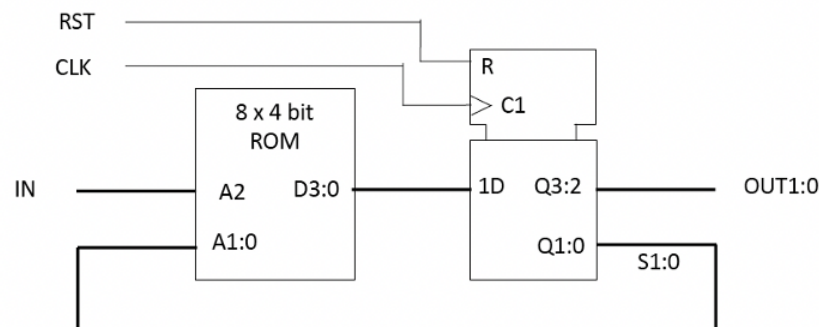


Figure 2.1

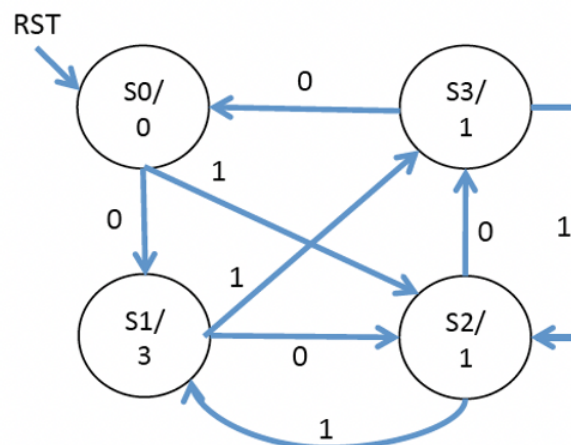
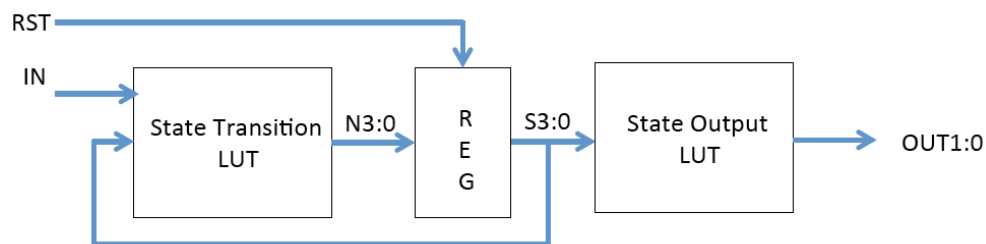


Figure 2.2

2. (a) This tests student's ability to analysis a FSM designed using ROM. [8]

ROM Address A2:0 (IN S1:0)	ROM Data D3:0
0 00	11 01
0 01	01 10
0 10	01 11
0 11	00 00
1 00	01 10
1 01	01 11
1 10	11 01
1 11	01 10

- (b) This tests student's ability to design with a FPGA logic element and map a FSM to it. [12]



Note that one-hot state encoding makes the design much easier. The states are now encoded as 0001, 0010, 0100, and 1000. The Boolean equations for N3:0 and OUT1:0 could be almost written down directly from the state diagram!

$$N0 = S3 * !IN + !S0 * !S1 * !S2 * !S3 \quad (2 \text{ LEs})$$

(the 2nd term is not necessary, ensure all zero state will go back to state 0)

$$N1 = S0 * !IN + S2 * IN \quad (1 \text{ LE})$$

$$N2 = S1 * !IN + S0 * IN + S3 * IN \quad (1 \text{ LE})$$

$$N3 = S2 * !IN + S1 * IN \quad (1 \text{ LE})$$

$$OUT0 = S1 + S2 \quad (1 \text{ LE})$$

$$OUT1 = S3$$

Assuming that N0 has the 2nd term, each LE can implement an arbitrary 4-input function (including the register). Therefore total LEs = 6. (OUT1 does not need any logic).

- (c) This test student's ability to specify a FSM using Verilog. [10]

```

module Q2c_FSM (OUT, IN, CLK, RST);
    input    IN, CLK, RST;
    output   [1:0] OUT;
    // define states
    parameter NSTATE = 4;
    parameter S_0 = 4'b0001;
    parameter S_1 = 4'b0010;
    parameter S_2 = 4'b0100;
    parameter S_3 = 4'b1000;

    reg [NSTATE-1:0] state;
    wire [1:0] OUT;
    wire          IN, RST, CLK

```

```

// specify state machine
always @ (posedge CLK)
  if (RST == 1'b1) begin
    state <= S_0;
    OUT <= 2'b00; end
  else
    case (state)
      S_0: begin
        OUT <= 0;
        If (IN == 1'b0)
          state <= S_1;
        else state <= S_2;
      end
      S_1: begin
        OUT <= 3;
        If (IN == 1'b0)
          state <= S_2;
        else state <= S_3;
      end
      S_2: begin
        OUT <= 1;
        If (IN == 1'b0)
          state <= S_3;
        else state <= S_1;
      end
      S_3: begin
        OUT <= 2;
        If (IN == 1'b0)
          state <= S_0;
        else state <= S_2;
      end
    else // defensive coding
      state <= S_0;
      OUT <= 0;
    end
  end
endmodule

```

3. *Figure 3.1a* shows the block diagram of a circuit that produces a pair of sinusoidal signals exactly $\pi/2$ radians apart (they are known as quadrature signals). The circuit consists of a dual-port 1024 x 9-bit ROM, a 10-bit binary counter, a binary adder and a clock divider circuit as shown. The clock divider divides the 50MHz system clock by a factor of K. The binary adder adds a constant value N to the counter output $\text{cnt}[9:0]$. The ROM stores the coefficient values for one complete cycle of a sinewave in 2's complement form and has a Verilog HDL interface shown in *Figure 3.1b*.

a) Explain how this circuit works.

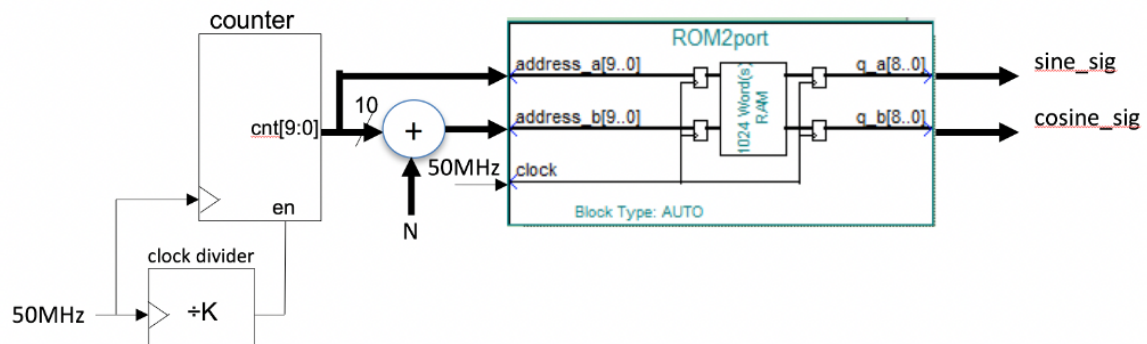
[10]

b) If you are required to produce the sine and cosine signals at a frequency closest to 10kHz, determine the value of N and K.

[5]

c) Implement in Verilog HDL, the circuit shown in *Figure 2.1a*. State to 4 significant digits the frequency of the output signals.

[10]



(a)

```

module ROM2port (
    address_a,
    address_b,
    clock,
    q_a,
    q_b);

    input [9:0] address_a;
    input [9:0] address_b;
    input clock;
    output [8:0] q_a;
    output [8:0] q_b;
endmodule

```

(b)

Figure 3/

Q3. This question tests student's ability to apply what they have learned in the Lab to a practical problem in sine and cosine generation.

- (a) Dual-port ROM consists of two independent ports, with separate address and data buses. In this way, the contents of the ROM can be assessed by two sources at two addresses simultaneously. Since the ROM contains 1024 samples of a complete sinewave, we can produce a sinewave by sequentially reading the contents of the ROM, increasing the address value using a 10-bit counter at each counter clock cycle. The quadrature signal (i.e. the sinewave that is 90 degrees apart) can be produce by offsetting the address value by $\frac{1}{4}$ of a cycle or 256 location.

[10]

- (b) The frequency of the sine and cosine signals is given by:

$$f_{sig} = f_{samp}/1024 = (50\text{MHz}/K) / 1024.$$

For f_{sig} to be as close to 10kHz as possible, we need f_{samp} to be 10.24MHz. Therefore the closest we can get to is to divide the 50MHz clock by 5.

Hence $K = 5$ and $N = 256$.

[5]

- (c)

```
module quadrature_gen (CLK50, sine_sig, cosine_sig);
    input CLK50; // 50 MHz clock
    output [8:0] sine_sig; // sinewave signal
    output [8:0] cosine_sig; // cosinewave signal

    // ---- clock divider
    reg [2:0] clk_ctr; // count clock cycles
    reg time_out; // goes high for 1 cycle every 4 clk cycles
    initial clk_ctr = 6'b0;
    parameter tc = 4; // count from 0 to 4
    always @ (posedge CLK50)
        if (clk_ctr==0) begin
            time_out <= 1'b1;
            clk_ctr <= tc;
        end
        else begin
            time_out <= 1'b0;
            clk_ctr <= clk_ctr + 1'b1;
        end
    // ---- end of clock divider

    // ---- address counter ----
    reg [9:0] address_a; // address counter for sine_sig
    reg [9:0] address_b; // address counter for cosine_sig
    initial address_a = 10'b0;

    always @ (posedge CLK50)
        if (time_out == 1'b1) begin
            address_a <= address_a + 1'b1;
            address_b <= address_a + 10'd256;
        end

    ROM2port sine_rom (address_a, address_b, CLK50, sine_sig, cosines_sig);
endmodule
```

[10]

Signal frequency = 9766Hz to 4 sf digits.